

Sesión 12 — Agentes de IA: el criterio de dónde y cuánta autonomía

Módulo 3 · Duración 120 min · Modalidad flipped classroom. Estado de entrada: los alumnos traen del ejercicio pre-sesión **un agente hecho**

a mano ya funcionando (no se reconstruye en el directo): el bucle manual sobre la

Responses API, las **tools planas** `strict:true`, los schemas y la traza `STEP N`, y el CLI `scripts/run_agent_s12.py`. Ese agente es **one-shot**: sobre un transcript, descompone → busca → calcula → valida **de un tiro, sin pausa humana**. Traen al directo su **traza de ejecución** y las dudas del ejercicio.

Estado de salida: los alumnos han ejercido el **criterio** que da título a la sesión — han **reencauzado** ese agente one-shot para que **sirva al flujo de estimación** (`Rag::EstimationRun`, el wizard de S10/S11) **respetando sus dos puertas humanas**, en vez de atropellarlas. El agente conduce las **dos fases** del wizard vía dos endpoints

(`POST /v1/estimate/agent/structure` y `POST /v1/estimate/agent/hours`, `app/api/routers/estimate_agent.py`), componiendo `agentic` + `rag` en el conductor (`app/domain/agent_estimation.py`). Y saben **por qué** cada pieza de la arquitectura del agente está donde está y **cómo influye** en el flujo que ya tenían.

Idioma: la prosa de esta guía está en español; **todo el código, payloads, nombres de campo, logs y prompts del LLM están en inglés** (convención del repo).

Nomenclatura: *servicio IA* = backend Python (`estimator/`); *backend de negocio* = Rails (`estimator-web/`). Nunca “backend” a secas.

Cómo leer este guion

Cada bloque usa los mismos marcadores que las guías de S10/S11, más uno nuevo (■ **Criterio**):

Marcador	Qué es	Dónde
■ Criterio	la pregunta de decisión humana que el bloque responde (el hilo de la sesión)	—
■ Fundamento · pizarra	la idea que se explica de viva voz	pizarra / slide
■ Dónde vive	archivos exactos (<code>ruta::función</code>) a abrir en pantalla	IDE
■ Demo · terminal	el comando cURL/CLI que se ejecuta en directo	terminal
■ UI en Rails (wizard)	dónde lo ve el usuario en el backend de negocio	navegador
■ Defensa oral / FAQ	lo que preguntarán y la respuesta corta	—
■ Contingencia	qué hacer si algo falla en el momento	—
■ Bridge	la frase puente al siguiente bloque	—

Tesis de la sesión. Un agente **no es una arquitectura que se “elige en bloque”** frente a un pipeline. Es una **decisión humana de dos niveles**: (1) ¿aporta aquí una decisión *adaptativa*, y **para qué** exactamente? (2) si aporta, ¿le doy el **volante entero** (one-shot) o **acoto su autonomía** a mi flujo y a mis puertas humanas? Hoy ejercemos ese criterio sobre un caso real: traéis un agente **one-shot** del pre-ejercicio; lo

reencauzamos para que sirva al flujo S11 sin atropellarlo. El bucle a mano —schema, `call_id`, tools— es la **mecánica** (la repasamos rápido); el **criterio de dónde y cuánta autonomía** es la lección.

Caso de anclaje. `estimator/exercises/session-12/sample_transcript_complex.txt` — el proyecto logístico **RUTA**, con **cuatro componentes que no tienen nada que ver entre sí** (backend de negocio greenfield + integración con SAP + app móvil de repartidores + panel de analítica). Es el hilo de toda la sesión: es exactamente el tipo de transcripción donde una **decisión adaptativa aporta** (nivel 1 del criterio) — y aun así **no** justifica darle al agente el volante entero (nivel 2).

Nota de coherencia con los artículos y el resto del repo. Todo el repo hace las llamadas LLM con `LLMWrapper` (Instructor + LiteLLM). El agente de S12 es **la única excepción deliberada**: conduce la **Responses API cruda** (`client.responses.create` / `.parse`) a mano, porque **ver el bucle es el objetivo del ejercicio**. No lo “arregles” para que use `LLMWrapper`. Menciónalo una vez y sigue.

Prerrequisitos (montar 10–15 min antes)

```
cd /Users/antonioperez/projects/ia/ai-engineering/estimator

# 1. Levantar el stack (aplica migraciones; el servicio IA queda en :8000).
docker compose up -d --build

# 2. Corpus de tareas históricas: es lo que search_budgets recupera (chunk_type=historical_task).
# Sin esto el agente busca y no encuentra análogos.
docker compose exec estimator python scripts/build_task_corpus.py --ingest
```

Variables de entorno relevantes (`.env`) — las de S12 son **plain settings** (no hay endpoint de runtime-config esta sesión; el CLI y el request las sobrescriben por-llamada):

```
OPENAI_API_KEY=...           # el agente usa la Responses API de OpenAI específicamente
ESTIMATE_API_KEY=demo-estimate-key # header X-API-Key del pipeline de estimación (y del agente)
AGENT_MODEL=gpt-5            # orquestador (config.py); el request/CLI lo puede pisar
AGENT_REASONING_EFFORT=medium # reasoning effort del orquestador
AGENT_MAX_ITERATIONS=10      # salvaguarda del bucle (una iteración = un round-trip)
AGENT_SEARCH_TOP_K=5          # lo que search_budgets pasa a retrieve()
AGENT_SEARCH_DISTANCE_THRESHOLD=0.6 # umbral de distancia coseno de la recuperación
```

Exporta para abreviar los cURL del directo:

```
export EST=demo-estimate-key
export B=http://localhost:8000
```

Trampa de entorno conocida (macOS Apple Silicon). El agente **real** (no-stub) llama a `retrieve()`, que es pgvector **asíncrono**. En el host puede fallar con `greenlet required`.

Por eso el agente real se corre **por el endpoint** (dentro del contenedor) o el CLI dentro del contenedor; el **CLI** `--stub` **sí corre en el host** (no toca la BBDD). Y si tu shell tiene `OPENAI_API_KEY` exportada, **pisa** la del `.env`: `unset OPENAI_API_KEY` o corre dentro del contenedor.

Pre-flight checklist (cURL en frío, dentro del Bloque 0)

Si las 5 filas pasan, el directo está listo.

#	Qué verifica	Esperado
1	Servicio IA vivo	<code>{"status":"healthy",...}</code>
2	Corpus de tareas ingerido (search_budgets tiene análogos)	<code>> 1500</code>
3	Endpoints del agente registrados y autenticados	<code>401</code> sin clave, <code>200</code> con clave
4	El bucle está cableado (offline, sin coste ni BBDD)	traza <code>STEP N</code> en pantalla
5	Los tests del agente en verde	<code>passed</code>

```
# 1 – servicio IA vivo
curl -s $B/health | jq -c

# 2 – corpus de tareas ingerido (docker exec por nombre de contenedor)
docker exec estimator-postgres psql -U estimator -d estimator \
  -c "select count(*) from budget_chunks where chunk_type='historical_tasks';"

# 3 – el endpoint de fase 1 exige la clave de estimación (401 sin ella)
curl -s -o /dev/null -w "%{http_code}\n" -X POST $B/v1/estimate/agent/structure \
  -H "Content-Type: application/json" -d '{"query":{"function":"x"}}'
# → 401

# 4 – el bucle, offline y barato: LLM real + recuperación enlatada (reference_retrieval.py), sin BBDD.
# El CLI corre las DOS fases: estructura (fase 1) → recuperación de horas (fase 2).
uv run python scripts/run_agent_s12.py exercises/session-12/sample_transcript_simple.txt --stub

# 5 – tests network-free del agente (bucle + tools + conductor + endpoints)
uv run pytest tests/generation/agent tests/domain/test_agent_estimation.py tests/api/test_estimate_agent.py -q
```

Regla innegociable. El bucle se conduce **a mano** (no delegamos en el encadenado interno de la Responses API): reason → act → observe → repite, con una salvaguarda de `max_iterations`. Las horas las calcula `derive_task_hours` con el **consenso ponderado por distancia determinista** de S10 (`rag/task_hours.py`) — el agente **decide la búsqueda**,

nunca la aritmética. Y el agente **entra en el flujo del wizard sin cambiarlo**: mismos pasos, mismas puertas humanas, mismos contratos de respuesta. Los frameworks (LangChain / LangGraph) son la S13; aquí no aparecen.

Bloque 0 — Encuadre: la decisión, no la magia (10 min)

[SLIDE: La pregunta no es "¿pipeline o agente?" — es "¿volante entero o autonomía acotada a mi flujo?"]

■ Criterio (el hilo del bloque)

- La pregunta que abre y cierra la sesión: **¿le doy el volante entero al agente (one-shot) o acoto su autonomía al flujo que ya tengo?** Hoy la respondemos con código.

■ Fundamento · pizarra (3 min)

- Recap mínimo (el alumno ya trae la teoría): un agente = **bucle**. Un modelo que **decide** qué tool llamar, unas tools que **ejecutan**, una **condición de parada**. Eso es la *mecánica* — la repasamos en el Bloque 1 en 10 minutos.
- Lo importante hoy es el **criterio de dónde ponerlo**. Un agente **no compite** con el pipeline fijo de S9–S11 en abstracto: la pregunta útil es **en qué punto** de *mi* flujo una decisión adaptativa gana su sueldo, y **cuánta** libertad le doy allí.

■ Demo · escenificar el ANTES (one-shot) (4 min)

- Proyecta una **traza one-shot**: la committeada `exercises/session-12/example_trace_complex.txt` o, mejor, la que un alumno trajo del pre-ejercicio. Léela en voz alta: descompuso en cuatro componentes, lanzó `search_budgets`, calculó y **te dio un número — de un tiro, sin que nadie revisara la estructura ni confirmara las horas**.
- **Funciona**. Y sin embargo: en nuestro negocio la **estructura la aprueba un humano** (¿son esos los módulos y tareas?) y las **horas/tarifas se confirman** (¿este número es defendible ante el cliente?). Un agente one-shot **atropella** esas dos puertas y produce un artefacto que **no encaja** en el flujo ni en sus contratos. **No es que el agente sea malo — es que le dimos el volante cuando no tocaba**.

[SLIDE: La traza one-shot — un número de un tiro, sin puerta humana. Funciona, pero no encaja]

■ Fundamento · el caso RUTA (3 min)

- Proyecta `sample_transcript_complex.txt`: el cliente **avisa** — "es grande, hay varias piezas que no tienen nada que ver entre sí": **backend de negocio** (greenfield), **integración SAP** (IDocs, reintentos), **app móvil** (offline), **panel de analítica** (KPIs).
- Aquí una decisión adaptativa **sí aporta** (nivel 1 del criterio): recuperar análogos para SAP y para una app móvil y para analítica con **una sola** query es justo lo que un pipeline fijo hace mal. Pero eso **no** implica volante entero (nivel 2): lo resolvemos **acotando** al agente al flujo. Cómo, en el Bloque 2.

■ Bridge → Bloque 1

- "Antes de decidir *dónde* ponerlo, repasemos rápido las **piezas móviles** del agente — las que ya depurasteis. En 10 minutos, y al Bloque 2, que es el corazón de hoy."

Bloque 1 — Anatomía del bucle (repaso rápido) (10 min)

[SLIDE: Las piezas móviles que en el Bloque 2 vamos a COLOCAR en el flujo]

■ Criterio (el hilo del bloque)

- Estos cuatro puntos son la **mecánica** del agente. Conocerlos bien es requisito para el criterio de después: no puedes decidir *dónde* acotar una autonomía que no entiendes.

Repaso ágil de los cuatro fallos frecuentes del pre-sesión, contra el código real. Una línea + dónde vive + un test. Demos offline con `--stub`.

■ Error 1 — schema plano (Responses) vs anidado (Chat Completions) · 3 min

■ **Dónde vive.** `app/generation/agentive/agent_tools.py` — `SEARCH_BUDGETS_TOOL`,
`DERIVE_TASK_HOURS_TOOL`, `VALIDATE_ESTIMATE_TOOL` (`HOURS_TOOL_SCHEMAS`; la fase 1 no usa tools
→ `STRUCTURE_TOOL_SCHEMAS = []`).

- Responses API = schema **plano** (`{"type": "function", "name": "...", "parameters": {...}}`), no anidado bajo `"function"`. Con `strict: true`: **todo** en `required`, opcional = **nullable** (`["string", "null"]`), `additionalProperties: false` en cada nivel. Míralo en `filters`.

```
# app/generation/agentive/agent_tools.py — SEARCH_BUDGETS_TOOL
SEARCH_BUDGETS_TOOL = {
    "type": "function",          # PLANO: name/description/parameters al mismo nivel
    "name": "search_budgets",    # (Chat Completions anida todo bajo la clave "function")
    "description": "...",
    "parameters": {
        "type": "object",
        "properties": {
            "query": {"type": "string", "description": "..."},
            "filters": {          # opcional → NO se omite: unión nullable
                "type": ["object", "null"],
                "properties": {
                    "sectors": {"type": ["array", "null"], "items": {"type": "string"}},
                    "component_type": {"type": ["string", "null"]},
                },
                "required": ["sectors", "component_type"], # strict → TODO en required
            },
            "additionalProperties": False,
        },
        "required": ["query", "filters"],
        "additionalProperties": False,
    },
}
```

```
"strict": True,
}
```

Qué muestra: la forma plana + `strict:true` fuerza que cada propiedad esté en `required` y que la opcionalidad se modele con `["<tipo>", "null"]`, con `additionalProperties:false` en cada nivel.

[SLIDE: Responses API = schema PLANO + strict:true (todo required, opcional = nullable)]

■ Error 2 — el `call_id` olvidado al devolver el resultado · 3 min

■ **Dónde vive.** `agent_loop.py::run_task_hours_recovery_agent` — el `function_call_output` con `call.call_id`; test `test_agent_loop.py::test_recovery_call_ids_are_echoed_back`.

- Cada resultado vuelve como `{"type":"function_call_output","call_id":call.call_id,"output":json.dumps(result)}`. `call_id` que no coincide → el modelo no sabe a qué llamada responde. `output` debe ser **string**.

```
# app/generation/agentive/agent_loop.py – devolver cada resultado de tool
tool_outputs.append({
    "type": "function_call_output",
    "call_id": call.call_id,          # el MISMO id que envió el modelo
    "output": json.dumps(result),    # string JSON, no un dict
})
# ... y re-llamar encadenando el estado en el servidor (previous_response_id):
response = await client.responses.create(
    model=model, previous_response_id=response.id,
    input=tool_outputs, tools=HOURS_TOOL_SCHEMAS, ...)
```

Qué muestra: el resultado se ata a su llamada por `call_id` y se serializa a string; la continuación reusa `previous_response_id` (el servidor conserva el orden de reasoning/tool-calls).

```
uv run pytest tests/generation/agentive/test_agent_loop.py::test_recovery_call_ids_are_echoed_back -q
# → 1 passed (la 2ª llamada re-envía {"s1","s2"} como function_call_output)
```

■ Error 3 — asumir una sola tool por vuelta · 2 min

■ **Dónde vive.** `agent_loop.py` — `_function_calls(response.output)` **itera**.

- Un turno puede pedir **varias** tools. Recorre TODO `response.output`, ejecútalas todas, devuelve **todos** los `function_call_output`. Se ve en la fase 2: varias `search_budgets` en un turno (una por tarea sin análogo), marcadas "parallel tool call in the same turn as STEP N".

```
# app/generation/agentive/agent_loop.py – un turno puede pedir VARIAS tools
def _function_calls(output):
    return [item for item in output if getattr(item, "type", None) == "function_call"]

calls = _function_calls(response.output) # TODAS, no solo la primera
```

```
for call in calls:                                # ... ejecuta cada una y acumula
    su salida
    result = await dispatch_tool(call.name, json.loads(call.arguments),
                                backend=retrieval_backend, consensus_fn=c
onsensus_fn)
    tool_outputs.append({"type": "function_call_output",
                        "call_id": call.call_id, "output": json.dumps(res
ult)})
```

Qué muestra: quedarse con la primera `function_call` rompe el bucle; hay que iterar todas las del turno y devolverlas juntas.

■ Error 4 — descripciones de tools vagas · 2 min

■ **Dónde vive.** Las `description` de `SEARCH_BUDGETS_TOOL` / `DERIVE_TASK_HOURS_TOOL`.

- La descripción es **lo único** que el modelo lee para decidir cuándo/cómo usar la tool. Es un **teaser**: la convertimos en ejercicio antes/después en el 2.3.

```
# app/generation/agentive/agent_tools.py – la description ES el contrato qu
e lee el modelo
"description": (
    "Search historical project tasks for work analogous to ONE task... "
    "Use a focused, task-specific query – reformulate it ... if the first se
arch finds nothing."
),
```

Qué muestra: una frase ("una por tarea", "reformula si no encuentras") es lo que dirige la decisión del modelo — el detalle completo, en el 2.3.

■ Bridge → Bloque 2

- "Piezas claras. Ahora la decisión de la sesión: teníais un agente one-shot; vamos a **acotarlo al flujo** — y a ver cómo cada pieza de arquitectura sirve a esa decisión."

Bloque 2 — Casos avanzados (95 min)

Sub-bloque 2.1 — El criterio hecho arquitectura: one-shot vs acotado al flujo (40 min)

[SLIDE: Del volante entero (one-shot) a la autonomía acotada: dos fases + gate humano + híbrido]

■ Criterio (el hilo del bloque)

- **Nivel 1 — ¿aporta y para qué?** Sí: RUTA es heterogéneo, y hay dos puntos del flujo donde una decisión adaptativa gana: **descomponer** el proyecto y **recuperar** análogos cuando la búsqueda por defecto falla.
- **Nivel 2 — ¿volante entero o acotado? Acotado.** No cambiamos el flujo ni tocamos las puertas humanas: metemos al agente **dentro** de los dos pasos donde aporta, y **nada más**.

■ Fundamento · pizarra — el contraste (10 min)

- **ANTES (one-shot, lo que traíais):** `run_estimation_agent` sobre el transcript crudo → descompone + busca + calcula + valida **de un tiro** → `AgentEstimate`. **Autonomía total, cero puertas humanas.** (Es la traza del Bloque 0.)
- **El problema de criterio:** el flujo S10/S11 (`Rag::EstimationRun`) tiene **dos gates humanos** — se revisa la estructura (paso «Revisión») y se confirman horas/tarifas (paso «Validación») — y **contratos** propios (`Estimate` / `TaskHoursResult`). El one-shot los ignora.
- **La decisión (DESPUÉS, acotado):** partimos la autonomía del agente en los **dos puntos** donde aporta, con el **gate humano en medio** y un paso de horas **híbrido**:

```
transcript → reformulación → estructura (AGENTE, fase 1) → REVISIÓN HUMANA
→ horas (determinista + recuperación del AGENTE, fase 2) → VALIDACIÓN HUMANA
```

[SLIDE: One-shot atropella dos puertas humanas · Acotado las respeta y aporta solo donde toca]

■ Dónde vive · la arquitectura, pieza a pieza, mapeada al flujo S11 (15 min)

Abre estos ficheros y recórrelos: **cada pieza es una decisión de acotar autonomía.**

1 · Dos entrypoints, uno por fase — `app/api/routers/estimate_agent.py`

```
@router.post("/structure", response_model=GenerateResult, ...) # FASE 1
async def structure(request, payload: AgentStructureRequest):
    return await agent_propose_structure(payload.query, client=_require_async_client(), ...)

@router.post("/hours", response_model=TaskHoursResult, ...) # FASE 2
async def hours(request, payload: AgentHoursRequest):
    return await agent_estimate_task_hours(payload.modules, client=_require_async_client(), ...)
```

→ Parten al agente en los **dos puntos** del flujo donde aporta, con el **gate humano** entre medias. Un endpoint = una decisión acotada, **no** el proyecto entero de un tiro (antes: un solo `/agent/run`).

2 · Contratos reutilizados — `app/generation/rag/schemas.py`

```
class GenerateResult(BaseModel):
    estimate: Estimate
    # ...
    agent_trace: AgentTrace | None = None # ← única adición

class TaskHoursResult(BaseModel):
    tasks: list[TaskHoursEstimate] = Field(default_factory=list)
    agent_trace: AgentTrace | None = None # ← única adición
```

→ El agente **devuelve la misma forma** que el camino determinista; el wizard la parsea **sin cambiar ni una vista**. Solo crece con `agent_trace` (opcional). El agente sirve al flujo, no al revés.

3 · El conductor — “cuándo el agente”, hecho código — `app/domain/agent_estimation.py`

```
async def agent_estimate_task_hours(modules, *, client, ...):
    base = await estimate_all(modules, ...) # 1) DETERMINISTA primero
    flagged = [AgentTaskRef(...) for est in base.tasks # 2) solo las que fallan
                if _flag_reason(est)]
    if not flagged or client is None: # coste CERO en el caso feliz
        return TaskHoursResult(tasks=base.tasks, agent_trace=AgentTrace())
    run = await run_task_hours_recovery_agent(flagged, ...) # 3) el AGENTE sobre las flagged
    # 4) merge: sobrescribe solo lo que el agente logró fundamentar

def _flag_reason(est): # el criterio de "cuándo"
    if not est.has_match: return "no historical analog..."
    if est.hours_range is not None: return "historical analogs contradictory..."
    if est.reliability is not None and est.reliability < _LOW_RELIABILITY:
        return f"low reliability ({est.reliability})"
    return None
```

→ Aquí vive el criterio: **determinista primero**, el agente entra **solo** en las tareas flagged (sin match / rango contradictorio / baja fiabilidad). Si no hay ninguna, ni se invoca.

4 · Backend + consensus inyectados — qué decide y qué NO — `agent_tools.py` + `agent_estimation.py`

```
# agent_tools.py – el agente decide la búsqueda; la aritmética NO es del modelo
def derive_task_hours(raw_args, *, consensus_fn):
    args = DeriveTaskHoursArgs.model_validate(raw_args)
    pairs = [(n.estimated_hours, n.distance) for n in args.neighbors]
    hours, reliability, dispersion = consensus_fn(pairs) # ← consenso S10 inyectado
    return {"module": ..., "task": ..., "estimated_hours": hours, "reliability": reliability, ...}

# agent_estimation.py – el conductor cablea las piezas de rag DENTRO del bucle
run = await run_task_hours_recovery_agent(flagged,
    retrieval_backend=make_retrieval_backend(top_k, distance_threshold),
    # rag/agent_retrieval.py
```

```
consensus_fn=distance_weighted_consensus, ...)
# rag/task_hours.py (S10)
```

→ Acota **qué** decide el agente (la **búsqueda**) y qué **no** (la **aritmética** sigue siendo el consenso determinista de S10). `agent_tools` solo conoce tipos estructurales (`ConsensusFn` , `RetrievalBackend`) → ningún hermano importa al otro.

5 · Fase 1 sin tools — razona pero no busca — `app/generation/agentive/agent_loop.py`

```
async def run_structure_agent(brief, *, client, model, ...):
    response = await client.responses.parse(          # .parse, NO .create
    con tools=
        model=model, instructions=STRUCTURE_SYSTEM_PROMPT,
        input=[{"role": "user", "content": brief}],
        reasoning={"effort": reasoning_effort, "summary": "auto"},
        text_format=AgentStructure,                  # salida estructurad
    a, sin herramientas
        store=True,
    )
    structure = response.output_parsed
```

(`STRUCTURE_TOOL_SCHEMAS = []`) → El agente **razona y decompone, pero no recupera** — respeta la decisión de S10 de no fundamentar la estructura en presupuestos (empobrecía el árbol).

6 · La traza como contrato compartido — `app/domain/schemas/agent_trace.py`

```
class AgentStep(BaseModel):
    step: int; reasoning_summary: str | None; tool: str; tool_args: dict;
    observation: str

class AgentTrace(BaseModel):
    steps: list[AgentStep] = Field(default_factory=list)
    def render(self) -> str: ...          # formato STEP N

# app/generation/rag/schemas.py la importa DESDE domain → rag lleva la tra
za sin tocar agentive:
from app.domain.schemas.agent_trace import AgentTrace
```

→ La autonomía acotada queda **auditable** dentro del flujo, y `rag` puede transportar la traza **sin importar** `agentive` (la traza vive en `domain` , que ambos pueden importar — regla de capas).

Nota al mentor — el conductor es el único punto de composición. `agentive` y `rag` son hermanos de `generation` y **no se importan entre sí**; se encuentran **solo** en `domain/agent_estimation.py` (regla de `ARCHITECTURE.md`). De paso, mover el backend de retrieval de `agentive` a `rag/agent_retrieval.py` arregló una fuga de capas del pre-ejercicio.

Nota al mentor — coste cero en el caso feliz. Si el pase determinista encuentra análogo para cada tarea, **el agente ni se invoca** (traza vacía). El agente se gana el sueldo **solo** donde el pipeline fijo se rinde — eso es “acotar” hecho coste.

■ Demo · terminal — las dos fases + el determinista como comparación (10 min)

```
# FASE 1 — el agente propone la estructura (sin horas, sin fuentes). Guardamos los módulos.
STRUCT=$(curl -s -X POST $B/v1/estimate/agent/structure -H "X-API-Key:$EST" -H "Content-Type: application/json" \
  -d '{"query":{"function":"Plataforma logística RUTA: backend de negocio, integración SAP, app móvil de repartidores y panel de analítica","sector":"logistics","technologies":["Rails","React Native"]}}')
echo "$STRUCT" | jq '{confidence:.estimate.confidence,
                    modules:[.estimate.modules[].name],
                    trace_steps:(.agent_trace.steps|length)}'

# FASE 2 — (tras la revisión humana) horas híbridas. Pasa los módulos/tareas revisados.
MODS=$(echo "$STRUCT" | jq '{modules:[.estimate.modules[] | {name, tasks:[.tasks[] | {name, description}]}]}')
curl -s -X POST $B/v1/estimate/agent/hours -H "X-API-Key:$EST" -H "Content-Type: application/json" \
  -d "$MODS" \
  | jq '{tasks:(.tasks|length),
        sin_dato:([.tasks[]|select(.has_match==false)]|length),
        recuperacion:(.agent_trace.steps|map(.tool)|group_by(.)|map({[0]):length})|add}'}

# → las tareas con buen match salen del determinista; las raras disparan la recuperación del agente.

# Comparación: el mismo paso de horas SIN agente (determinista puro).
curl -s -X POST $B/v1/estimate/tasks/hours -H "X-API-Key:$EST" -H "Content-Type: application/json" \
  -d "$MODS" | jq '{tasks:(.tasks|length), sin_dato:([.tasks[]|select(.has_match==false)]|length)}'
```

- **Léelo, no lo cuentes de memoria:** el determinista deja tareas raras **sin dato**; el híbrido las **recupera** reformulando la búsqueda (mira `agent_trace`). Mismo contrato (`TaskHoursResult`) que el determinista — el agente añade la **capa de decisión**, no la aritmética.

■ UI en Rails — consola «Agentes» (5 min)

[SLIDE: Consola «Agentes»: perfiles con nombre + persona que alimentan las DOS fases]

- **Dónde vive.** Nav «Agentes» → `/agents/profiles` (`Agents::ProfilesController`); modelo `Agents::Profile` (name + `config` JSONB + persona + `is_default`).

```
# app/models/agents/profile.rb — el perfil → overrides por-llamada a AMBAS fases
def config_payload
```

```

{ "model" => model, "reasoning_effort" => reasoning_effort,
  "max_iterations" => max_iterations&.to_i, "search_top_k" => search_top_k&.to_i,
  "search_distance_threshold" => search_distance_threshold&.to_f }.compact
# blanco → default del servicio
end

```

- **Demo.** Crea un perfil «Veloz» (`gpt-5-mini` / effort `low` / persona “ve al grano”). Ese `config_payload` se envía como overrides **por-llamada** a `/agent/structure` y `/agent/hours` — el servicio IA sigue siendo el dueño del comportamiento; el backend de negocio solo guarda presets.

■ Criterio (cierre del bloque)

- **Acotar no es mutilar.** El agente sigue decidiendo lo que **solo él** puede hacer bien (re-buscar y reformular lo raro); lo demás lo deciden el **flujo** (el orden de pasos) y el **humano** (los dos gates). Esa es la respuesta de nivel 2.

■ Defensa oral / FAQ

- “¿Por qué no reusa `LLMWrapper` como todo lo demás?” → Excepción deliberada del ejercicio: Responses API cruda para **ver** el bucle.
- “¿No es más caro que el determinista?” → Solo corre sobre las tareas que fallaron, y ni se invoca si no hay ninguna. Convierte **banderas rojas** en horas fundamentadas.
- “¿Por qué no dejarlo one-shot y ya?” → Porque atropella los dos gates humanos y rompe los contratos del wizard. El one-shot **funciona**, pero no **encaja** — y encajar es el requisito de negocio.

■ Contingencia

- Si la fase 2 peta con `greenlet required`, es el host: córrela por el **endpoint** (contenedor).
- Si nada se recupera / `total` raro: falta el corpus de tareas → `build_task_corpus.py --ingest`.

■ Bridge → 2.2

- “Hemos acotado al agente por diseño. Ahora la **prueba** de que se quedó dentro de sus límites: su traza. Abramos la caja.”

Sub-bloque 2.2 — La traza como evidencia de la decisión acotada (28 min)

[SLIDE: La traza no es un log — es la prueba de que el agente decidió DENTRO de sus límites]

■ Criterio (el hilo del bloque)

- Acotar una autonomía sin poder **auditarla** es un acto de fe. La traza es lo que convierte “confía en que el agente se portó” en “míralo: decidió esto, aquí, y nada más”.

■ Fundamento · pizarra (4 min)

- La traza es el **artefacto de auditoría** del agente. Formato `STEP N` con tres líneas:
`reasoning` (resumen de razonamiento), `action` (`tool(args)`), `observation` (lo que devolvió la tool). Reason → act → observe, hecho legible.

■ Dónde vive · el formato y las dos trazas (10 min)

- El render:** `app/domain/schemas/agent_trace.py::AgentTrace.render()` produce el `STEP N`.
`reasoning={"effort":..., "summary": "auto"}` en el loop es lo que hace que la Responses API emita resúmenes de razonamiento.

```
# app/domain/schemas/agent_trace.py – AgentTrace.render() → el formato STEP N
for s in self.steps:
    blocks.append(
        f"STEP{s.step}\n"
        f"  reasoning:{s.reasoning_summary or '(no reasoning summary emitted)'}\n"
        f"  action:{s.tool}({args})\n"          # args = tool_args como JSON
        f"  observation:{s.observation}"
    )
```

```
# app/generation/agent/agent_loop.py – el razonamiento se atribuye al TURNO, las horas al TOOL
reasoning_summary = _extract_reasoning_summary(response.output) # gpt-5 razona 1 vez por turno
step_reasoning = (reasoning_summary if step_no == first_step_in_turn
                  else f"(parallel tool call in the same turn as STEP{first_step_in_turn})")
# ... las horas recuperadas salen del TOOL determinista, no del modelo:
if name == "derive_task_hours" and "error" not in result:
    derivations[key] = AgentTaskDerivation(
        module=..., task=...,
        estimated_hours=result.get("estimated_hours"),
        reliability=result.get("reliability"),
        has_match=bool(result.get("has_match", False)))
```

Qué muestra: el resumen de razonamiento pertenece al turno (se marca en las tools paralelas), y la captura de `derive_task_hours` es la prueba de que el número lo pone la **tool**, no el modelo.

- **Traza de fase 1 (estructura):** fina, un paso `propose_structure` con el resumen y “N módulos / M tareas”. No hay tools: es un `responses.parse`. **La prueba:** el agente decidió la estructura **y no buscó** — exactamente el límite que le pusimos.
- **Traza de fase 2 (recuperación):** el reason → act → observe de verdad. Recórrela:
- **Decide/reformula** (STEP 1...N): por cada tarea sin análogo, `search_budgets`; si el filtro de sector devolvió **cero**, el agente **amplía** (re-lanza con `filters.sectors: null`) — el momento que el pase determinista **no** hace.
- **Deriva** (`derive_task_hours`): consenso ponderado por distancia **determinista** (S10). El número es reproducible, **no** inventado. **La prueba:** el agente eligió la búsqueda, no el

número.

- **Valida** (`validate_estimate` , último paso): guardarraíles al estilo S4/S5.

Aha. `validate_estimate` son **guardarraíles al estilo S4/S5** (el ACB de `boss.py` / `critic.py` que ya conocéis), pero aquí es **una tool que el agente decide llamar**, no un orquestador fijo. Misma disciplina, distinta forma.

Antes/después con las trazas. La committeada `exercises/session-12/example_trace_complex.txt` es del flujo **one-shot** (verás `calculate_estimate` y un `AgentEstimate` de un tiro): úsala como **exhibit del "ANTES"**. Compárala con una corrida **nueva** del CLI de dos fases (`scripts/run_agent_s12.py ... --out ...`): fase 1 (estructura) + fase 2 (recuperación). El contraste de trazas **es** el contraste de criterio.

■ UI en Rails (wizard) — la traza en los pasos «Estructura» y «Horas» (14 min)

La traza en `jq` es para nosotros; el usuario la ve **dentro** de los dos pasos del wizard, no en una pantalla aparte.

• Dónde vive (cliente):

- Controlador: `app/controllers/rag/estimation_runs_controller.rb` — `generate` (fase 1) y `estimate_hours` (fase 2) llaman al agente con el perfil elegido; la **puerta humana** entre medias (`review` → `estimate_hours`) **no cambia**.

```
# app/controllers/rag/estimation_runs_controller.rb – el agente conduce la
s dos fases
def generate                                     # FASE 1
  payload = rag_client(...).agent_generate_structure(query:, config: profile
&.config_payload, persona: profile&.persona)
  @run.update!(generation: payload, structure: seed_structure(payload), cu
rrent_step: "review")
end

def estimate_hours                               # FASE 2 (tras el
gate humano "review")
  result = rag_client(...).agent_estimate_task_hours(modules: structure_for_
api(modules), config:, persona:)
  @run.update!(task_hours: result, adjusted_breakdown: seed_breakdown_with
_hours(modules, result), current_step: "hours")
end
```

- POROs: `Rag::AgentTraceView` (→ `Rag::AgentStepView`). `Rag::EstimationRun` gana `generation_agent_trace` / `hours_agent_trace`, que leen `agent_trace` de los JSONB de fase (**sin columna nueva**):

```
# app/models/rag/estimation_run.rb – la traza viaja dentro del JSONB de ca
da fase
def generation_agent_trace
  raw = generation["agent_trace"] if generation.present?
```

```

Rag::AgentTraceView.from_hash(raw) if raw.present?
end

def hours_agent_trace
  raw = task_hours["agent_trace"] if task_hours.present?
  Rag::AgentTraceView.from_hash(raw) if raw.present?
end

```

- Vistas: el partial compartido `_agent_trace.html.erb` (STEP N tipado) se renderiza bajo la estructura (`_step_generation`) y bajo las horas (`_step_hours`); `_agent_profile_select` va en los forms de «Estructura» y «Revisión».

```

<%= app/views/rag/estimation_runs/_agent_trace.html.erb - la misma trazaSTEP N, tipada%>
<% trace.steps.eachdo|s|%>
  <% tool_classes=case s.tool
when"search_budgets"then"text-brand-soft"
when"derive_task_hours"then"text-success-light"
when"propose_structure"then"text-brand"
  else"text-white/70"end%>
  <span>STEP <%= s.step%></span> · <span class="<%= tool_classes%>"><%= s.tool%></span>
  <p>reasoning: <%= s.reasoning%></p>
  <p>action: <%= s.action%></p>      <%= s.action="tool(args)"%>
  <p>observation: <%= s.observation%></p>
<%end%>

```

• Paseo en el navegador (`/rag/estimation_runs`):

1. Crea una ejecución pegando la transcripción RUTA (`sample_transcript_complex.txt`) → reformulación.
2. **Estructura:** elige el perfil «**Veloz**» (consola «Agentes», 2.1) → **Generar estructura**. El agente decompone; su **traza fina** aparece bajo el árbol.
3. **Revisión (gate humano):** edita/añade módulos y tareas → **Estimar horas por tarea**.
4. **Horas:** el determinista rellena las tareas con análogo; la **traza de recuperación** (STEP N: `search_budgets` reformulando → `derive_task_hours`) muestra cómo el agente recuperó las tareas raras (o "no tuvo que recuperar ninguna" si todas tenían análogo).

Nota de coherencia. El wizard S10/S11 sigue **intacto** en pasos y contratos; el agente es ahora **cómo se computan** la estructura y las horas, respetando las dos puertas humanas.

■ Defensa oral / FAQ

- “¿El agente puede quedarse en bucle?” → No: `max_iterations` corta y `stopped_reason` lo dice. Es la salvaguarda sobre la parada natural (un turno sin `function_call`).
- “¿Y si una tool falla?” → El error vuelve como el `output` de esa tool (string) y el modelo **se autocorrig**e; no se cae el bucle (test `test_recovery_bad_tool_args_do_not_crash_the_loop`).

■ Contingencia

- Si «Estructura» u «Horas» tardan y el spinner parece colgado: es el `reasoning` de gpt-5; el timeout es de 300 s. Si excede, reintenta o baja `reasoning_effort` .

- Si la traza de recuperación sale vacía: el determinista encontró análogo para cada tarea (caso feliz, no un bug). Enséñalo con una tarea rara a propósito.

■ Bridge → 2.3

- “Ya sabéis leer que el agente decidió dentro de sus límites. Lo último: **cómo dirigir** esa decisión sin tocar el modelo — con una frase en la descripción de una tool.”

Sub-bloque 2.3 — Dirigir la decisión del agente: descripciones de tools (27 min)

■ Criterio (el hilo del bloque)

- Acotar al agente a un rol (2.1) no basta: **dentro** de ese rol sigue decidiendo. La **descripción de la tool** es la palanca barata y localizada para dirigir esa decisión.

■ Fundamento · pizarra (4 min)

- Iteramos **barato y offline** con `-stub`: LLM real (decide de verdad) + recuperación enlatada (`reference_retrieval.py`, sin BBDD ni coste). Cambiamos una `description`, re-corremos, y **leemos la traza**. El servicio IA bind-montea `./app`: editar `agent_tools.py` se recoge sin rebuild.

■ Demo · antes/después nº1 — `search_budgets` (13 min)

■ Dónde vive. La `description` de `SEARCH_BUDGETS_TOOL` en `agent_tools.py`.

- **Antes (vaga)**. Sustituye por algo genérico, quitando la guía de enfoque y de reformulación:
 - > `"Search historical budgets and return matching items with their hours."`

```
uv run python scripts/run_agent_s12.py exercises/session-12/sample_transcript_complex.txt --stub --effort low
```

Síntoma en la traza de **recuperación**: búsquedas **genéricas** y **sin reformular** → deja tareas sin recuperar que sí tenían análogo con otra query.

- **Después (precisa)**. Restaura la descripción real, que **enseña a enfocar y a reformular**. El agente pasa a **una búsqueda enfocada por tarea** y **re-busca** cuando un filtro no da nada.

```
# app/generation/agent/agent_tools.py - SEARCH_BUDGETS_TOOL["description"] (real)
"Search historical project tasks for work analogous to ONE task you are trying to cost..."
"Call this once per task you need to ground. Use a focused, task-specific query - "
"reformulate it (different wording, synonyms, or drop/relax a filter) if the first search "
```

```
"finds nothing. ... pass those (hours + distance) as the neighbors for derive_task_hours."
```

Qué muestra: "una por tarea" + "reformula si no encuentras" es lo que convierte una búsqueda fallida en una recuperación exitosa — sin tocar el modelo.

[SLIDE: "reformula si no encuentras" — una frase que cambia cuántas tareas recupera]

■ Demo · antes/después n°2 — `derive_task_hours` (9 min)

■ Dónde vive. La `description` de `DERIVE_TASK_HOURS_TOOL`.

- **Antes.** Quita `"This does NOT call a model – it is pure arithmetic..."` y la guía de pasar los vecinos con su `distance`. Síntoma: el modelo tiende a **estimar horas él mismo** o a llamar sin las distancias, en vez de delegar el consenso determinista.
- **Después.** Restaura la frase: el agente **recoge** los vecinos (`hours + distance`) de `search_budgets` y deja el consenso a la tool. El número es **reproducible**, no inventado.

```
# app/generation/agent/agent_tools.py – DERIVE_TASK_HOURS_TOOL["description"] (real)
"Deterministically derive the engineer-hours for ONE task from the historical analogs you "
"found with search_budgets. It computes a distance-weighted consensus... Pass the neighbours "
"exactly as search_budgets returned them (each with its estimated_hours AND its distance). "
"This does NOT call a model – it is pure arithmetic, so its output is reproducible..."
```

Qué muestra: nombrar el contrato ("es aritmética, no un modelo" + "pasa los vecinos con su distance") mueve al agente de *inventar* horas a *delegar* el consenso determinista.

[SLIDE: Nombrar el contrato de la tool ("es aritmética, no un modelo") cambia a quién delega]

■ Defensa oral / FAQ

- "¿Esto no es prompt engineering disfrazado?" → Exactamente, pero en el sitio correcto: la **interfaz de la tool**. Optimizar ahí es barato y localizado.
- "¿Reproducible con `-stub`?" → El modelo varía, pero el **patrón** (reformular vs no; delegar vs inventar) es estable en 1–2 corridas.

■ Contingencia

- **Restaura** las descripciones reales antes de cerrar (o `git checkout app/generation/agent/agent_tools.py`), para no dejar el repo en la versión "antes".

■ Bridge → Cierre

- "Habéis decidido dónde poner al agente, cómo acotarlo, cómo auditarlo y cómo dirigirlo. Una frase sobre lo que os lleváis."

Bloque 3 — Cierre: el criterio, transferible (5 min)

■ Fundamento · el arco

- El meta-skill de hoy, en dos preguntas que os lleváis a **cualquier** agente: **(1)** ¿aporta aquí una decisión adaptativa, y **para qué?** **(2)** ¿le doy el **volante entero** o **acoto** su autonomía a mi flujo y mis puertas humanas? El reflejo por defecto **no** es dar el volante entero.
- El caso S12 fue la respuesta hecha código: un agente one-shot **reencauzado** a las dos fases del wizard, con el gate humano intacto, la aritmética determinista y la traza auditable. **El comportamiento agéntico se adaptó al flujo que queríamos, no al revés.**

■ Entrega

- Rama de referencia `session_12_live` : servicio IA (`agent_loop.py` / `agent_tools.py` / `agent_schemas.py` + conductor `domain/agent_estimation.py` + router `estimate_agent.py`); backend de negocio (el agente en los pasos «Estructura» y «Horas» del wizard).

■ Bridge → Sesión 13

- "Todo lo que hoy escribimos a mano —el bucle, el despacho de tools, el `call_id`, la parada— la próxima sesión lo delega un **framework** (LangChain / LangGraph). Pero fijaos: el framework facilita la **mecánica**; **el criterio de dónde y cuánta autonomía sigue siendo humano**. Hoy lo habéis ejercido a mano para que mañana sepáis qué decide el framework por vosotros y qué **no** debéis delegarle."

Apéndice ejecutivo (para el mentor)

Resumen por bloque

#	Bloque	Min	Demo clave
0	Encuadre: la decisión	10	traza one-shot (el "antes") + RUTA; ¿volante entero o acotado?
1	Anatomía del bucle (repaso)	10	schema plano · <code>call_id</code> · varias tools/turno · descripciones (<code>test_agent_loop.py</code>)
2.1	El criterio hecho arquitectura	40	one-shot vs acotado + tabla arquitectura → flujo + <code>agent/structure</code> / <code>agent/hours</code> vs determinista
2.2	La traza como evidencia	28	traza fase 1 (estructura) vs fase 2 (recuperación); "antes" (one-shot) vs "después"; wizard
2.3	Dirigir la decisión (descripciones)	27	antes/después de <code>search_budgets</code> y <code>derive_task_hours</code> con <code>--stub</code>
3	Cierre: el criterio, transferible	5	rama <code>session_12_live</code> ; handover a S13 (frameworks)

Total: 10 + 10 + (40 + 28 + 27) + 5 = **120 min.**

Criterio de decisión — checklist proyectable (el corazón de la sesión)

Antes de meter un agente en cualquier flujo, cinco preguntas:

1. **¿Aporta una decisión adaptativa?** ¿El input es heterogéneo/impredecible, o un pipeline fijo ya lo resuelve? (RUTA: sí aporta; un formulario cerrado: no.)
2. **¿Para qué exactamente?** Nombra el/los punto(s) concretos (aquí: descomponer y re-buscar lo raro), no “para estimar” a secas.
3. **¿Qué NO debe decidir el agente?** La **aritmética** (consenso determinista), los **contratos** y las **puertas humanas**. Acótalo fuera de eso.
4. **¿Dónde van las puertas humanas?** Si el flujo ya las tiene, el agente las **respeto**, no las sustituye.
5. **¿Volante entero o acotado?** Por defecto, **acotado**: el agente decide lo que solo él hace bien; el flujo y el humano deciden el resto. La respuesta de S12.

Tabla de errores previsibles

Síntoma	Causa	Arreglo
401 en <code>agent/structure</code> o <code>agent/hours</code>	falta/no coincide <code>ESTIMATE_API_KEY</code> (header <code>X-API-Key</code>)	alinea <code>.env</code> del servicio IA con <code>\$EST</code>
Fase 2 peta <code>greenlet required</code>	host Apple Silicon + <code>retrieve()</code> async	córrela por el endpoint (contenedor); el CLI <code>--stub</code> sí va en host
Nada se recupera / <code>total</code> raro	corpus de tareas sin ingerir	<code>docker compose exec estimator python scripts/build_task_corpus.py --ingest</code>
El agente no reformula y deja tareas sin recuperar	descripción de <code>search_budgets</code> vaga	restaura la descripción real (guía “reformula si no encuentras”)
El modelo inventa horas	descripción de <code>derive_task_hours</code> sin “es aritmética, no un modelo”	restaura la frase; delega el consenso en la tool
<code>stopped_reason:</code> <code>max_iterations</code> en la recuperación	bucle no converge / <code>max_iterations</code> bajo	sube <code>max_iterations</code> (perfil) o baja effort; lee la traza
<code>OPENAI_API_KEY</code> del shell pisa <code>.env</code>	export en la terminal	<code>unset OPENAI_API_KEY</code> o corre en el contenedor

Artefactos nuevos / modificados

Artefacto	Rol
<code>app/api/routers/estimate_agent.py</code>	<code>POST /v1/estimate/agent/structure + /hours</code> (auth <code>ESTIMATE_API_KEY</code> , 15/min, <code>response_model</code> reutilizado <code>GenerateResult / TaskHoursResult</code>)
<code>app/domain/agent_estimation.py</code>	nuevo · conductor: <code>agent_propose_structure</code> (fase 1) + <code>agent_estimate_task_hours</code> (fase 2 híbrida) + <code>_flag_reason</code> . Único punto donde <code>agentic</code> + <code>rag</code> se componen
<code>app/domain/schemas/agent_trace.py</code>	nuevo · <code>AgentStep / AgentTrace</code> (contrato de traza compartido; lo lleva

Artefacto	Rol
	<code>GenerateResult</code> / <code>TaskHoursResult</code>)
<code>app/generation/agentic/agent_loop.py</code>	<code>run_structure_agent</code> (fase 1, sin tools) + <code>run_task_hours_recovery_agent</code> (fase 2)
<code>app/generation/agentic/agent_tools.py</code>	<code>search_budgets</code> + <code>derive_task_hours</code> (consenso S10 inyectado) + <code>validate_estimate</code> ; tipos estructurales (sin importar <code>rag</code> / <code>dependencies</code>)
<code>app/generation/agentic/agent_schemas.py</code>	<code>AgentStructure</code> / <code>AgentTaskRef</code> / <code>AgentTaskDerivation</code> / <code>AgentTaskHoursRun</code> (se retiró la familia one-shot)
<code>app/generation/rag/agent_retrieval.py</code>	nuevo · <code>make_retrieval_backend</code> / <code>default_retrieval_backend</code> (movido desde <code>agentic</code> , arregla capas)
<code>app/generation/rag/schemas.py</code>	<code>agent_trace</code> opcional en <code>GenerateResult</code> / <code>TaskHoursResult</code> ; <code>AgentStructureRequest</code> / <code>AgentHoursRequest</code>
<code>tests/*</code>	<code>test_agent_loop</code> · <code>test_agent_tools</code> · <code>test_agent_retrieval</code> · <code>test_agent_estimation</code> (conductor) · <code>test_estimate_agent</code> (endpoints)
<code>exercises/session-12/example_trace_complex.txt</code>	exhibit “ANTES (one-shot)” — traza del flujo retirado; se conserva a propósito para el contraste, no se regenera
(cliente) <code>app/services/estimator_ai/rag_estimate_client.rb</code>	<code>#agent_generate_structure</code> + <code>#agent_estimate_task_hours</code>
(cliente) <code>estimation_runs_controller.rb</code> · <code>config/routes.rb</code>	<code>generate</code> / <code>estimate_hours</code> llaman al agente + perfil; se retira <code>run_agent</code>
(cliente) <code>app/models/rag/estimation_run.rb</code>	se retira el paso/columna <code>agent</code> ; <code>generation_agent_trace</code> / <code>hours_agent_trace</code>
(cliente) <code>_agent_trace.html.erb</code> · <code>_agent_profile_select.html.erb</code>	nuevos · panel STEP N + selector de perfil, en «Estructura» y «Horas»
(cliente) <code>app/models/rag/agent_trace_view.rb</code> · <code>agent_step_view.rb</code>	POROs de traza (se retiraron <code>agent_run/estimate/component_view</code>)
(cliente) consola «Agentes» (<code>agents/profiles_controller.rb</code> · <code>Agents::Profile</code> · vistas)	perfiles nombrados del agente S12, aplicados como overrides por-llamada a ambas fases

Settings de S12 (plain settings — sin runtime-config esta sesión)

`AGENT_MODEL` (`gpt-5`), `AGENT_REASONING_EFFORT` (`medium`), `AGENT_MAX_ITERATIONS` (`10`),
`AGENT_SEARCH_TOP_K` (`5`), `AGENT_SEARCH_DISTANCE_THRESHOLD` (`0.6`) — en `config.py` . El request,
el CLI y **los perfiles de la consola «Agentes»** las **sobreescriben por-llamada** (`model` ,
`reasoning_effort` , `max_iterations` , `search_top_k` , `search_distance_threshold` y una `persona`
que se añade al system prompt), lo que permite comparar la misma entrada a distinto
coste/comportamiento sin recrear el contenedor.

Límites de alcance (no negociables)

- El bucle se conduce **a mano; sin frameworks** (LangChain / LangGraph = S13).

- Las horas las calcula `derive_task_hours` con el **consenso determinista** de S10 — el agente decide la búsqueda, nunca la aritmética.
- La traza de razonamiento se **expone** en la respuesta (`agent_trace` en `GenerateResult` / `TaskHoursResult`).
- **El agente se acota al flujo:** entra en el wizard sin cambiar pasos, puertas humanas ni contratos, y en horas es **híbrido** (determinista → recuperación del agente). **No se le da el volante entero por defecto** — ese es el criterio de la sesión.